Ders 26 Genişletilmiş Notu: Sepet Analizinde Üçlüler, Kumarbaz Problemi Simülasyonu, Ortanca Filtresi ve Resim Boyutlandırma

Ders 26 Genişletilmiş Notu: Sepet Analizinde Üçlüler, Kumarbaz Problemi Simülasyonu, Ortanca Filtresi ve Resim Boyutlandırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sepet Analizi Tekrarı: A ve T Matrisleri
- 2. Ürün Çiftlerinden Ürün Üçlülerine
- 3. İkili Sürümün Karmaşıklığı
- 4. Üç Boyutlu Matrisle Ürün Üçlüleri Kodu
- 5. if mi Çarpım mı: Hangisi Daha Uzun Sürer?
- 6. Sonucu Yazdırmak ve Okumak
- 7. En Çok Birlikte Satılan 10 Ürün ve Apriori
- 8. Olasılıksal Algoritma: Kumarbaz Problemi
- 9. Tuz-Biber Gürültüsü: Ortalama Filtresi ve Öğrenci Önerileri
- 10. Ortanca (Median) Filtresi
- 11. Resmi Küçültme
- 12. Resmi Büyütme
- 13. Ayna Görüntüsü

1. Sepet Analizi Tekrarı: A ve T Matrisleri

Bu derste önceki derslerde yarım kalan örnekler tamamlandı: ürün üçlüleri, olasılıksal bir algoritma ve görüntü işleme örnekleri. Hoca önce sepet analizini kısaca hatırlattı. Elimizdeki soru şuydu: Hangi ürün hangi ürünle birlikte satılmış?

Başlangıçta karışık bir liste vardır. Bu listeden, hangi ürünün hangi sepette bulunduğunu gösteren 0-1 değerli bir `A` matrisi elde edilir. Örneğin “bir numaralı ürün iki ve üç numaralı sepette var” bilgisi bu matristen okunur.

Burada ürünler grup olarak düşünülür. Hoca bunu şöyle anlattı: Kalemse kalemdir. Falanca markanın kalemi, filanca markanın kalemi diye ayrılmaz. Alt kategorilere bölersek pilot kalem, dolma kalem, kurşun kalem ayrımı yapılabilir, ama burada yapmıyoruz. Domatesse de organik domates, salkım domates diye düşünmeyiz. Domates domatestir, marka ayrımına girmeyiz.

`A` matrisinden bir `T` matrisi elde ediliyordu. `T`, ürünlerin birlikte kaç kez satıldığını gösterir. Aynı ürünün kendisiyle olan hücresi sıfır kalır. Örneğin “2 numaralı ürün 1 numaralı ürünle 23 kez satılmış” gibi bir bilgi bu matristen okunur.

İkili sürümde üç döngü vardı ve `A[i][k]` üzerinde geziliyordu. Mantık şöyleydi:

1. `i` ile birinci ürünün satırı taranır.
2. Bu ürünün hangi sepette olduğuna bakılır. Bir sepette yoksa o sepet geçilir.
3. Ürün bir sepette varsa aynı sepette sonraki ürünler tek tek kontrol edilir.

4. Sonraki ürün de o sepette varsa T'de iki ürünün kesiştiği hücre 1 artırılır.
5. Sonra birinci ürünün bulunduğu bir sonraki sepete geçilir.

2. Ürün Çiftlerinden Ürün Üçlülerine

Üçlüleri bulmak için ikili sürümle bir benzerlik kurulur. `A[i][k]`, i. ürünün k. sepetteki durumudur. Diyelim ki 1 numaralı ürün 2 numaralı sepette var. Aynı sepette 4 numaralı ürünü de bulduk. Şimdi soru şudur: Bu ikisiyle beraber bu sepette başka hangi ürünler var?

Bunun için bir döngü daha gerekir. Birinci ürünü bulduktan sonra, aynı sepette onun indisinden bir sonraki ürünlere bakılır. İkinci ürün de bulununca ondan sonraki ürünlere bakılır. Birinci üründe sıfır görseydik, o sepette onunla beraber başka bir şey aramazdık.

Bu aramanın sonucunu saklamak için yeni bir boyut gerekir. Birinci ürün satırı, ikinci ürün sütunu gösterir, üçüncü ürün ise yüksekliği gösterir. Dolayısıyla üç boyutlu bir `T[i][j][k]` matrisine ihtiyaç vardır. İndisler birer birer ilerler: Birinci ürüne baktıysak ikinciyi ondan sonrakilerde, üçüncüyü de ikinciden sonrakilerde ararız. İlgili hücre, üç ürün de sepette varsa bir artar. Bunu `if` yazmadan üç hücrenin çarpımıyla da yapabiliriz: Üçü de 1 ise çarpım 1 olur, biri bile 0 ise çarpım 0 olur ve sayaç değişmez.

Üç koşulun `if` ile yazılmış karşılığı da aynı işi görür:

```
if (A[i][m] == 1 && A[j][m] == 1 && A[k][m] == 1) {
    T[i][j][k]++;
}
```

3. İkili Sürümün Karmaşıklığı

Hoca kodlamaya geçmeden önce, bir öğrencinin e-postayla ilettiği konuyu da açıklamak için, ikili sürümün sözde kodu üzerinden zaman ve bellek karmaşıklığını hesapladı.

3.1 Temel İşlem

Karmaşıklık hesabında önce "kesinlikle çalışacak" kod aranır. Burada bu, `if` içindeki karşılaştırmadır. Bu yüzden algoritmanın temel işlemi (basic operation), yani yükünü çeken işlem, karşılaştırma kabul edilir. Satırda iki `==` ve bir `&&` vardır, ama sabit sayıdaki işlemler tek işlem kabul edilir. Bu işlem kaç kez tekrarlanırsa algoritma o kadar karmaşıktır.

Satırın içindeki toplama ve atama da hesaba katılırsa sayı 4-5'e çıkar. Toplamanın, karşılaştırmanın ve atamanın maliyetleri farklıdır. Ama karşılaştırma genellikle en maliyetli işlemlerden biri olduğu için toplama ve atama göz ardı edilir. Göz ardı etmek de gerekir, çünkü atama her zaman yapılmaz, `if` ise her zaman çalışır. Hoca burada bu tek işleme "bir saniye" ya da "bir dakika" demediğini, "bir birim zaman" dediğini özellikle belirtti.

3.2 Zaman Karmaşıklığı

Hoca döngü sınırlarını $i = 1 \dots n$, $j = i + 1 \dots n$, $k = j + 1 \dots n$ diye okuyarak iç içe toplamı içeriden dışarıya çözdü:

- En içteki toplam: $\sum_{k=j+1}^{n} 1 = n - (j + 1) + 1 = n - j$.
- Ortadaki toplam: $\sum_{j=i+1}^{n} (n - j) = (n - i)\, n - \sum_{j=i+1}^{n} j$. Sağdaki j toplamı terim toplamı formülünden gelir ve n^2'li bir terim verir. Bütün ifade $a \cdot n^2$ biçiminde yazılabilir. a bir katsayıdır, küçük terimler atılır.
- Dıştaki döngü n kez döner: $n \cdot n^2 = n^3$.

Sonuç şudur: Ürün sayısı ve sepet sayısı n kabul edilirse, iç içe üç döngünün olabilecek üst sınırı n^3'tür. 10 ürün ve 10 sepet varsa bu karşılaştırma **en fazla** 10^3 kez yapılır.

> **Ders sonrası not:** Hoca üçüncü döngüyü $k = j + 1 \dots n$ diye okudu. Ama `A[i][k]` yapısında k sepet indisi ise, k büyük olasılıkla $j + 1$'den değil, ilk sepetten başlar. Python ile sayıldığında $n = 10$ için sözlü okuma $\binom{10}{3} = 120$, sepet döngüsünü baştan

başlatan sürüm $\binom{10}{2} \cdot 10 = 450$ karşılaştırma verir. İkisi de 10^3 üst sınırının altındadır, yani hocanın sonucu değişmez.

3.3 Üst Sınır Ne Demektir?

Hoca üst limit ve alt limit kavramlarının bazen karıştığını söyledi. Algoritmanın n değerine göre grafiği çizilirse bir n^3 eğrisi çıkar. Bu eğri, algoritmanın tam olarak o değerde çalışacağını söylemez. Algoritma eğrinin altındaki aralıkta bir yerde çalışır, ama çizgiyi hiçbir zaman geçmez. En iyi durum analizi de yapılabilir, ama burada ona girilmedi.

3.4 Bellek Karmaşıklığı

Bellekte üç matris vardır:

- Başlangıçtaki liste: n satır ve 2 sütun, yani $n \times 2$.
- `A` matrisi: n ürün ve n sepet için ayrıca n^2 yer.
- `T` matrisi: ürün sayısı çarpı ürün sayısı, yine ayrıca n^2 yer.

Toplam ek yer $2n^2$'dir. Yani algoritma n^2'lik ek bellek karmaşıklığına sahiptir.

3.5 if'li Kod Neden İyi Bir Koddur?

Hocaya göre `if`'li kod iyi bir koddur, çünkü algoritmanın içeriği n^3 kez çalışmayacaktır. `A` matrisinde bir sıfır varsa, örneğin birinci sepette 1 numaralı ürün yoksa, o sepete 1 numaralı ürün için hiç bakılmaz. n^3'ten en az bir sıfır için n kadar işlem çıkmış olur.

`if` yerine çarpım yazılırsa bütün matris kesinlikle dolaşılır ve n^3'e kesin ulaşılır. Koşul yazmadan kod yazma (conditional olmadan kod yazma) diye bir yaklaşım da vardır ve orada bu tür yazımlar kullanılır. Yani program `if`'siz de yazılabilir.

> **Hocanın vurgusu:** Olabilecek en üst sınır n^3'tür, ama döngüler ne kadar az çalışır, ne kadar erken biterse o kadar iyidir. Hoca bunu algoritma analizi dersinde daha ayrıntılı anlattığını, burada bu kadarının yeterli olduğunu söyledi.

4. Üç Boyutlu Matrisle Ürün Üçlüleri Kodu

4.1 Veri ve A Matrisinin Doldurulması

Hoca örnek bir sepet listesi yazdı. Liste, her biri iki sayılık 6 satırdır: (1, 2), (2, 1), (2, 4), (2, 2), (1, 4), (3, 3). Ürün sayısı 5, sepet sayısı 4 alındı. Numaralar sıfırdan başladığı için 0 numaralı ürün de vardır.

`A` ve `T` matrisleri sıfırlanmış başlasın diye `static` tanımlandı. `A` 10'a 10, `T` 10'a 10'a 10 boyutundadır.

`A` tek döngüyle doldurulur. Her satırın bir indisi hangi sepet olduğunu, diğer indisi hangi ürün olduğunu gösterir. Hoca burada öğrencilerin sık yaptığı bir yazımı da andı: `sepet` iki sütunlu olduğu için içeriye `j`'yi 0'dan 2'ye döndüren ikinci bir döngü yazıp indis olarak `j` kullanmak. Bu yanlış değildir, ama gerek de yoktur. İki-üç eleman varsa tek satırda yazmak yeterlidir, çünkü döngünün mikroişlemci tarafında ek bir yükü vardır ve programı biraz yavaşlatır.

4.2 Dört Döngü

Dış üç döngü ürünler üzerinde döner. `j`, `i+1`'den, `k` de `j+1`'den başlar. En içteki `m` döngüsü sepetleri tarar. Hoca, bir boyut ekleyince bir döngü de eklendiğini belirterek karmaşıklığı sınıfa sordu. Cevap n^4'tür.

Hoca bu dönem belki de ilk kez üç boyutlu bir matris üzerinde işlem yapıldığını da söyledi.

> **Ek bilgi (derste anlatılmadı):** Derste bellek hesabı yalnız ikili sürüm için yapıldı. Üç boyutlu `T` kullanıldığında ek bellek büyük olasılıkla n^2'den n^3 mertebesine çıkar, çünkü `T` artık ürün sayısının küpü kadar hücre tutar.

4.3 Big-O Hakkında Bir Söz

Hoca bu sırada Big-O gösterimine de değindi. Big-O bir üst sınır (upper bound) gösterimidir. $O(n^4)$ yazmak, algoritmanın olabilecek en kötü durumdaki karmaşıklığını ifade eder. Bu derste Big-O'nun ayrıntısı anlatılmadığı için, şimdilik "n^4" demek daha doğrudur. Omega ve Teta gösterimleri de vardır. Bunlar ayrık matematik, veri yapıları ve algoritma analizi derslerinde görülecek.

4.4 Tam Kod

Aşağıdaki program derste yazılan `Sepet3D` kodunun derlenmiş halidir.

```c
#include <stdio.h>

int main(void)
{
    /* Her satır: {sepet numarası, ürün numarası} */
    int sepet[10][2] = {{1, 2}, {2, 1}, {2, 4}, {2, 2}, {1, 4}, {3, 3}};
    int n = 6;              /* satır sayısı */
    int urun_sayisi = 5;    /* ürün numaraları 0-4 */
    int sepet_sayisi = 4;   /* sepet numaraları 0-3 */
    static int A[10][10];   /* static olduğu için sıfırlanmış başlar */
    static int T[10][10][10];
    int i, j, k, m;

    for (i = 0; i < n; i++) {
        A[sepet[i][1]][sepet[i][0]] = 1;
    }

    for (i = 0; i < urun_sayisi; i++) {
        for (j = 0; j < sepet_sayisi; j++) {
            printf("%3d", A[i][j]);
        }
        printf("\n");
    }

    for (i = 0; i < urun_sayisi; i++) {
        for (j = i + 1; j < urun_sayisi; j++) {
            for (k = j + 1; k < urun_sayisi; k++) {
                for (m = 0; m < sepet_sayisi; m++) {
                    T[i][j][k] = T[i][j][k] + A[i][m] * A[j][m] * A[k][m];
                }
            }
        }
    }

    for (i = 0; i < urun_sayisi; i++) {
        for (j = 0; j < urun_sayisi; j++) {
            for (k = 0; k < urun_sayisi; k++) {
                printf("%d %d %d adet = %d\n", i, j, k, T[i][j][k]);
            }
        }
    }
    return 0;
}
```

Program önce `A` matrisini yazar. Satırlar ürünleri (0-4), sütunlar sepetleri (0-3) gösterir:

```
  0  0  0  0
  0  0  1  0
  0  1  1  0
```

```
0  0  0  1
0  1  1  0
```

Ders sonrası not: Hoca listeyi yazarken ve çıktıyı okurken ürün ve sepet sırasını yer yer karıştırdı. Örneğin (1, 4) satırını "1 numaralı ürün 4 numaralı sepette" diye okudu. Oysa aynı derste bulunan (1, 2, 4) üçlüsü ve hocanın "birinci sepette 2 ve 4, ikinci sepette 1, 2 ve 4 var" diye yaptığı kontrol, satırların büyük olasılıkla {sepet, ürün} sırasında olduğunu gösteriyor. Bu sırayla derlenen kod da dersteki sonucu verdi. Yukarıdaki matriste 4 numaralı ürün 1 ve 2 numaralı sepetlerdedir.

5. if mi Çarpım mı: Hangisi Daha Uzun Sürer?

Hoca kodu yazdıktan sonra şu soruyu sordu: Bir zamanlayıcı kullansak hangi kod daha uzun sürer, `if`'li olan mı, çarpımlı olan mı? Çarpımlı sürümde n^3'e kesin ulaşılıyor, `if`'lide ise atama işlemleri n^3'ten az yapılıyor.

Soru: Alttaki (çarpımlı) kod daha uzun sürmez mi? Ama yukarıda da `if` var, daha fazla işlem yapıyoruz galiba. **Cevap:** `if`'li sürümde koşul doğruysa ek olarak toplama ve atama vardır, doğru değilse ek bir şey yoktur. Dizi erişimlerini de sayarak bakalım.

Hoca `if`'li sürümdeki temel işlemleri tek tek saydı:

1. Diziye erişim.
2. Karşılaştırma.
3. && (ve) işlemi.
4. Diziye erişim.
5. Karşılaştırma.
6. Diziye erişim.
7. Atama.
8. Diziye erişim.
9. Toplama.

Dokuz işlemden ilk 5'i "kesinlikle" yapılır. Son dördü yalnız koşul doğruysa yapılır. Çarpımlı sürümde ise 7 temel işlem vardır ve hepsi her zaman yapılır. Hocanın sayımı özetle şöyledir:

Sürüm	Kesin yapılan	En fazla	Toplam sayım
`if`'li	5 işlem	9 işlem	$5n^3$ ile $9n^3$ arası
Çarpımlı	7 işlem	7 işlem	$7n^3$

Ders sonrası not: Hoca 5 işlemin kesin yapıldığını söyledi. C'de && soldan sağa değerlendirildiği ve ilk koşul yanlışsa ikinci koşula bakılmadığı için, bu durumda yapılan işlem sayısı büyük olasılıkla 5'ten azdır. Bu, `if`'li sürümün lehine bir farktır, karşılaştırmanın sonucunu değiştirmez.

Atamalar göz önüne alınırsa `if`'li sürüm daha kısa, çarpımlı sürüm daha uzun sürer. Ama kesin bir şey söylenemez. `if`'in saniye cinsinden maliyeti çarpımlı sürüme göre daha yüksek olabilir ve bu işlemciden işlemciye değişir. Adresleme modları da burada rol oynar. Bunu anlamak için, başka program çalışmayan bir bilgisayarda diğer parametreleri sabit tutarak test yapmak gerekir.

Hocanın önerdiği deney şöyledir:

1. Kodun başına ve sonuna `clock` konur.
2. Kodun ikinci sürümü yazılır ve ona da `clock` konur.
3. İki sürüm de defalarca, örneğin 50 veya 100 kez çalıştırılır.
4. Süreler ölçülür, ortalamalarına bakılır ve istatistiksel olarak karşılaştırılır.

Süre ölçmek tek başına belirleyici değildir. Bilgisayar sabit olsa bile, başka bir program çalışmamalı ve işletim sistemi çok fazla müdahale etmemelidir. Karmaşıklığı polinom fonksiyonlarla yazmamızın sebebi budur.

Soru: `clock` kaç kez kullanılabilir? **Cevap:** `main` içinde istediğiniz kadar kullanabilirsiniz. Bir başlangıcı, bir bitişi olur. Yalnız başlangıç ve bitiş noktalarını dikkatli belirlemek gerekir.

> **Hocanın vurgusu:** Süre ölçümleri tam anlamıyla belirleyici değildir. Aynı koşullarda defalarca ölçüp ortalamaya bakın.

6. Sonucu Yazdırmak ve Okumak

Üç boyutlu matrisi yazdırmak için `i`, `j`, `k` üçü de sıfırdan ürün sayısına kadar dönen üç döngü kullanılır. Her satıra ürün numaraları ve adet bilgisi yazılır. Hoca, üç boyutlu matrisi ekrana yazmanın o kadar kolay olmadığını söyledi: Ürün sayısı 5 olduğu için $5 \times 5 \times 5 = 125$ satırlık bir çıktı çıkar. `T`'nin en büyük elemanının hangi `i`, `j`, `k`'da olduğunu bulursak en çok birlikte satılan üçlüyü de dolaylı yoldan bulmuş oluruz.

Çıktıda yalnızca 1, 2 ve 4 numaralı ürünlerin birlikte satıldığı görüldü. Hoca bunu veriyle karşılaştırdı: Birinci sepette 2 ve 4 var, üçlü oluşmuyor. İkinci sepette 1, 4 ve 2 var, yani bulunan tek üçlü budur.

Sonra veri değiştirildi: Birinci sepette 3 de olsun. Böylece birinci sepette 2, 3 ve 4 numaralı ürünler bulundu. Hoca, 2, 4 ve 3'ün birlikte satıldığı durumu görmek istediğini söyledi. Derlenen kodda (3, 3) satırı (1, 3) yapılınca çıktıda sıfırdan farklı yalnız şu iki satır kalıyor:

```
1 2 4 adet = 1
2 3 4 adet = 1
```

7. En Çok Birlikte Satılan 10 Ürün ve Apriori

Hoca soruyu büyüttü: En çok birlikte satılan 10 ürünü bulmak istersek bu yaklaşım işe yarar mı? Üçlüde karmaşıklık n^4 idi (üç ürün döngüsü ve bir sepet döngüsü). 10'lu grupta 10 ürün döngüsü ve bir sepet döngüsü olur, yani n^{11}'e gider. 10 boyutlu bir matris de açılabilir, kod olarak yazılır, ama algoritma çok kötü bir yere doğru gider.

Bu iş için veri madenciliğinde başka algoritmalar vardır. Hoca, bildiği kadarıyla Apriori algoritmasının bu analizi yaptığını, ileride seçimlik derslerde görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Apriori'nin temel fikri büyük olasılıkla şudur: Bir ürün grubu sık satılmıyorsa, onu içeren daha büyük gruplar da sık satılamaz. Böylece bütün olası grupları saymak yerine, küçük sık gruplardan büyük adaylar üretilir ve arama uzayı daralır.

8. Olasılıksal Algoritma: Kumarbaz Problemi

8.1 Problem

Her oyunda p olasılıkla 1 TL kazanılır, $1 - p$ olasılıkla 1 TL kaybedilir. Elinde n TL olan kişi ya m TL kazanıp ($n + m$'ye ulaşıp) sevinerek ya da parasını tüketip eve dönecektir. Bu kişinin sevinerek dönme olasılığı nedir?

Hoca senaryoyu şöyle kurdu: Kişinin 100 lirası var ve belirli bir miktar daha kazanmak istiyor. Hedef $h = n + m$ olur. Kumarhanede 500 gün boyunca oynayanların kazanma olasılığı 0.473 olarak gözlenmiş olsun. Ama bu 0.473'ün kime vuracağı belli değildir. Olasılığı kullanmak için 0-1 arasında rastgele bir sayı üretilir.

8.2 Tek Oyuncu Simülasyonu

Bir kişinin oyunu şu adımlarla ilerler:

1. Başlangıçta para n, hedef $h = n + m$ ve adım sayacı $t = 0$'dır.
2. $n < h$ ve $n > 0$ olduğu sürece devam edilir. Hedefe ulaşan veya parası biten kişi kumarhaneden çıkar.
3. 0-1 arasında rastgele bir sayı üretilir.
4. Sayı $1 - p$'den büyükse kişi kazanmıştır: n bir artar. Değilse n bir azalır.
5. t bir artırılır ve o anki para dizide `t` indisine yazılır.

t burada döngüyü kontrol eden bir değişken değildir, yalnız diziye yazmak için kullanılır. Dizinin çizdirilmesiyle kişinin parasının zaman içindeki grafiği elde edilir. Program yeniden çalıştırılırsa benzer bir grafik çıkar.

8.3 Çok Oyunculu Deney

Tek kişilik oyun tek başına cevap vermez. Ya bir kişi çok kez oynar ya da aynı anda çok kişi üzerinde aynı deney yapılır. Bu yüzden yukarıdaki döngü dışarıdan K kez tekrarlanır. Döngü her bittiğinde kişi ya hedefe ulaşmıştır ya da parası bitmiştir. Hedefe ulaşmışsa kazananların sayısı bir artar. Bütün deneyler bitince bu sayı K'ya bölünür.

Hocanın gösterdiği slaytta $n = 100$, $m = 10$ için farklı K değerlerinde 0.339, 0.341 gibi sonuçlar vardı. Kazanma olasılığı, olasılık analizinin sonunda şu ifadeden gelir ve hocaya göre K'dan da n'den de bağımsızdır:

$$\left(\frac{p}{1-p}\right)^m$$

Sonuç bu değerden de küçük kalır, yani ona ulaşmaz. Hoca buradan şu sonucu çıkardı: Kasa her zaman kazanır. Oynayanların bazısı kazanır, bazısı kaybeder. "Kazandıkça kaybetmek" lafının karşılığı da budur: Oyunu ne kadar tekrarlarsanız tekrarlayın, bu oran değişmez.

Hoca bunu yazı tura örneğiyle açıkladı. Yazı tura 10 kez atıldığında 10'unun da aynı gelmesi mümkündür, yani 10'a 0 bir sonuç çıkabilir. Ama 10 milyon kez atılırsa yaklaşık 5 milyon yazı, 5 milyon tura gelir ve frekans $1/2$'ye yaklaşır. Simülasyonda da deney sayısı arttıkça sonuç bu olasılığa oturur.

8.4 Kod

Hoca ikinci kısmın kodunu yazdı. Burada iki önemli uyarı yaptı:

- `srand(time(NULL))` başlangıç değerini (seed) bir kez beslemek içindir ve `main`'in başına yazılır. Çok fonksiyon kullanılsa bile onların içinde yeniden çağrılmaz. Hoca bunun sık yapılan bir hata olduğunu söyledi.
- n her denemede güncellendiği için ilk değerin ayrı bir değişkende saklanması ve her denemenin başında geri yüklenmesi gerekir. Bir öğrenci bu noktayı derste düzeltti: Atama, dış döngünün içine alındı.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    int n, m, h, i;
    int ilk_para;
    int K = 1000;       /* deney sayısı; derste kullanılan değer okunmuyor */
    int kazanma = 0;
    float p;

    srand(time(NULL));  /* main'in başında yalnız bir kez */

    scanf("%d %d", &ilk_para, &m);
    scanf("%f", &p);
    h = ilk_para + m;

    for (i = 0; i < K; i++) {
        n = ilk_para;   /* her denemede para başa döner */
        while (n < h && n > 0) {
            if (rand() / (float) RAND_MAX > 1 - p) {
                n++;
```

```
            } else {
                n--;
            }
        }
        if (n == h) {
            kazanma++;
        }
    }
    printf("%3.3f\n", (float) kazanma / K);
    return 0;
}
```

Hoca `RAND_MAX` için önce hangi başlık dosyasının gerektiğinden emin olamadı. Derleyicinin söyleyeceğini belirterek devam etti.

> **Ders sonrası not:** `RAND_MAX` büyük olasılıkla `limits.h` değil, `stdlib.h` içinde tanımlıdır. Yukarıdaki kod yalnız `stdlib.h` ile `gcc -Wall` altında uyarısız derlendi.

8.5 Sonuçlar

Hoca önce $n = 10$, $m = 100$, $p = 0.473$ girdi ve 0 çıktı. "Nasıl 0 çıktı?" diye şaşırdı, sonra girişlerin yer değiştirdiğini fark etti ($n = 100$, $m = 10$ olacaktı). Düzeltilince sonuç 0.335 çıktı. Program da hemen bitmedi, döngü belli bir süre döndü. Hoca bunun tekrar tekrar denenebileceğini ve aynı yazı turada olduğu gibi sonucun oturacağını söyledi.

Verilen değerlerle formülün hesabı:

- Verilen: $p = 0.473$, $1 - p = 0.527$, $m = 10$.
- Oran: $\dfrac{p}{1-p} = \dfrac{0.473}{0.527} \approx 0.8975$ (ara adım eklendi).
- Sonuç: $0.8975^{10} \approx 0.3392$.

Derste bulunan 0.335 ile slayttaki 0.339-0.341 bu değere yakındır.

> **Ders sonrası not:** Hoca sonucun n'den bağımsız olduğunu söyledi. Tam formül, $r = (1-p)/p$ olmak üzere büyük olasılıkla $P = \dfrac{1 - r^{n}}{1 - r^{n+m}}$ biçimindedir ve n'i içerir. Bağımsızlık büyük olasılıkla yalnız $p < 0.5$ ve n büyükken yaklaşık olarak geçerlidir. Python ile $n = 100$, $m = 10$ için tam formül 0.33923, yaklaşık formül 0.33924 verir. Aynı hesap, hocanın yanlışlıkla girdiği $n = 10$, $m = 100$ için tam formülde 1.33×10^{-5}, yaklaşık formülde 2.02×10^{-5} verir. Yani küçük n'de fark belirgindir ve bu kadar küçük bir olasılık, ekranda 0 görülmesini de büyük olasılıkla açıklar. Hocanın "sonuç bundan küçük kalır" sözü olasılığın kendisi için doğrudur. Slayttaki 0.341 gibi simülasyon sonuçları bunu rastgelelikten dolayı büyük olasılıkla az miktarda aşabilir.
>
> **Ders sonrası not:** 0.335 sonucu üç ondalıklıdır. $K = 100$ ile oran 0.33, 0.34 gibi iki ondalıklı çıkar, üçüncü basamak 0 olur. Bu yüzden derste K büyük olasılıkla 100 değildi.

9. Tuz-Biber Gürültüsü: Ortalama Filtresi ve Öğrenci Önerileri

9.1 Problemin Hatırlanması

Aradan sonra görüntü işlemeye dönüldü. Okunan görüntü bir matristir. Bazı hücreleri 0 (siyah), bazı hücreleri 255 (beyaz) olmuştur. Buna tuz-biber gürültüsü denir. Önceki derste bunu gidermek için ortalama alınmıştı: 3×3 veya 5×5'lik bir pencere resmin üzerinde kaydırılarak her konumda pencere içindeki değerlerin ortalaması hesaplanıyordu.

Sonuç tatmin edici olmamıştı. 3×3'te siyah ve beyaz noktalar tam kaybolmadı. Sebep, ortalamanın 9 hücrenin toplamından oluşması ve gürültünün de bu toplama dahil olmasıydı. Gürültüyü resmin bir parçası gibi düşünürsek hata alırız. Filtre 5×5'e büyütülünce gürültü

neredeyse kayboldu, ama resim çok bulanıklaştı (blur). Daha da büyütülünce gürültü hiç görünmedi, fakat resmin ayrıntıları ve hatları kayboldu.

9.2 Birinci Öneri: 0 ve 255'i Ortalamaya Katmamak

Hoca çözüm önerisi sordu. Bir öğrenci, gürültülü pikselin ortalamaya dahil olmaması gerektiğini söyledi. Öneri şuna dönüştü: Pencerede değeri 0 veya 255 olan piksel varsa onu toplama katmayalım.

Hoca önce bu koşulu yazıp toplamı yine 9'a böldü. Sonra sordu: 0 ve 255 dışındakileri topladık, peki toplamı neye böleceğiz? O hücrelerin dışındakilerin sayısına mı, 9'a mı? Doğrusu bir sayaç (c) kullanıp toplamı sayaca bölmektir.

Soru: 0 veya 255'e eşitken ortalama alması gerekmiyor muydu? **Cevap:** Hayır. Eşit değilse toplanacak. Yani oradaki piksel değeri alınır, 0 ve 255 dahil edilmez.

Hoca sayacın nerede sıfırlanacağını da tartıştı: Satır başında değil, her filtre konumunda sıfırlanmalıdır. Deneme sırasında bir yanlışlık da oldu: Filtre boyutu daha önceki denemeden 21 kaldığı için görüntü farklı çıktı, boyut 3 yapılınca sonuç düzeldi.

```
void ortalama_sayacli(int **resim, int **b, int n, int m, int filtre)
{
    int i, j, e, g, toplam, c;
    for (i = filtre / 2; i < n - filtre / 2; i++) {
        for (j = filtre / 2; j < m - filtre / 2; j++) {
            toplam = 0;
            c = 0;      /* her filtre konumunda sıfırlanır */
            for (e = -filtre / 2; e <= filtre / 2; e++) {
                for (g = -filtre / 2; g <= filtre / 2; g++) {
                    if (resim[i + e][j + g] != 0 && resim[i + e][j + g] != 255) {
                        toplam += resim[i + e][j + g];
                        c++;
                    }
                }
            }
            b[i][j] = toplam / c;
        }
    }
}
```

> **Ders sonrası not:** Hoca koşulu sözlü olarak "0'a eşit değil **veya** 255'e eşit değil" diye söyledi. `||` ile kurulursa koşul büyük olasılıkla her piksel için doğru olur, çünkü bir değer aynı anda hem 0 hem 255 olamaz. Bu yüzden doğrusu büyük olasılıkla &&'dir. Ayrıca penceredeki bütün pikseller 0 veya 255 ise `c` sıfır kalır ve `toplam / c` sıfıra bölme olur. Gerçek siyah bölgelerde bu durum büyük olasılıkla ortaya çıkar, bu yüzden uygulamada büyük olasılıkla `c > 0` kontrolü gerekir.

Sonuç önemli bir ders verdi. Tuz-biber gitti, ama resimdeki gerçek siyah ve beyaz bölgeler bozuldu. Hoca farenin göründüğü yeri, yani siyahların ve beyazların bulunduğu yerleri gösterdi. Resimde anormal olan noktalar (outlier) 0 ve 255'lerdir. Ama insanın saçı, yüzündeki gölgeler de zaten siyah veya beyazdır. 0 ve 255'i tamamen dışarı atmak resmin doğasına aykırıdır.

> **Hocanın vurgusu:** Öyle bir şey yapılmalı ki resmin kendi dokusu bozulmasın.

9.3 İkinci Öneri: Standart Sapma

Başka bir öğrenci standart sapmayı önerdi: Pencerede ortalamadan standart sapmanın belli bir katından uzak olan aykırı pikseller atılsın, ortadaki normal değerler alınsın. Hoca fikri beğendi ve piksellerin dağılımını çizerek gösterdi. Ortalamaya yakın pikseller resmin normal pikselleridir, uçta kalanlar ise outlier'lardır. Bu yöntem olabilir, ama daha kolay bir yöntem vardır.

10. Ortanca (Median) Filtresi

10.1 Fikir

Penceredeki 9 pikselin ortalamasını ve standart sapmasını bulmak yerine onları sıralayalım. Sıralandıktan sonra 0'lar başa, 255'ler sona gider. Başka 0 veya 255 varsa onlar da uçlara yığılır. Tam ortadaki elemanı alırsak gürültü işin içine girmez. Bu ortadaki elemana ortanca (median) denir ve bu yönteme ortanca filtreleme (median filter) adı verilir.

Ortalama filtresiyle arasındaki temel fark, burada toplama yapılmamasıdır. Yani $a + b + c + 0 + 255$ gibi bir toplam oluşmaz. Yapılan iş, sıralamak ve ortanca değeri almaktır.

10.2 Pencerenin Başlangıcı ve Sınırları

Filtreleme resmin (0, 0) noktasından başlayamaz. Pencere 3×3 ise ilk merkez (1, 1) noktasıdır, $3/2$ gibi düşünülür. Pencere 5×5 ise ilk merkez (2, 2)'dir. Tam sayı bölmesi kullanıldığı için `FW / 2` ve `FH / 2` doğrudan yazılabilir.

Aynı şekilde resmin sonuna da pencerenin yarısı kadar yaklaşılamaz. Böylece resmin genişliği ve yüksekliğinden pencerenin yarısı kadar bir kayıp oluşur. Hoca bunu düzeltecek teknikler olduğunu, ama burada görüntü işleme dersi yapmadığımızı ve bu kadarının yeterli olduğunu söyledi.

Filtre boyutları makro ile tanımlandı. Bu sayede piksel dizisi `int piksel[FW * FH]` biçiminde açılabilir. Hoca bunun derleme öncesinde yapılan bir şey olduğunu, klavyeden girilen bir değerle böyle yapılamayacağını, makroyla bir sabit yazmaktan farkı olmadığını belirtti.

10.3 Algoritma

Her piksel için yapılan iş şöyledir:

1. Merkez `i` ve `j` belirlenir.
2. İki iç döngüyle pencere taranır. `fx` ve `fy`, 0'dan pencere boyutuna kadar döner.
3. Pencere içinde gerçek koordinat hesaplanır: `imx`, `i`'den `FW / 2` geri gidip `fx` kadar ilerlemiş noktadır. `imy` için aynı işlem yapılır.
4. Bu pikselin değeri diziye yazılır, dizi indisi bir artar.
5. Pencere bitince dizi sıralanır.
6. Ortadaki eleman, yani `piksel[FW * FH / 2]` yeni matrise yazılır.

Pencere dizisinin indisi (`t`) her yeni merkezde sıfırlanır. Ayrıca `count` değişkeni, iç döngülerin toplam kaç kez çalıştığını görmek için tutulur.

```
#define FW 3
#define FH 3

void resim_yaz(char *ad, int **b, int n, int m, int q);

void sirala(int *a, int n)
{
    int i, j, tmp;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
}

void median_filtre(int **resim, int **b, int n, int m, int q)
{
```

```
    int i, j, fx, fy, imx, imy, t;
    int piksel[FW * FH];
    int w = n - FW / 2;
    int h = m - FH / 2;
    int x0 = FW / 2;
    int y0 = FH / 2;
    int count = 0;
    char yeni_resim_adi[] = "lena_median_1.pgm";

    for (i = x0; i < w; i++) {
        for (j = y0; j < h; j++) {
            t = 0;
            for (fx = 0; fx < FW; fx++) {
                for (fy = 0; fy < FH; fy++) {
                    imx = i - FW / 2 + fx;
                    imy = j - FH / 2 + fy;
                    piksel[t] = resim[imx][imy];
                    t++;
                    count++;
                }
            }
            sirala(piksel, FW * FH);
            b[i][j] = piksel[FW * FH / 2];
        }
    }
    printf("%d\n", count);
    resim_yaz(yeni_resim_adi, b, n, m, q);
    printf("tamam\n\n");
}
```

Sıralama için hoca en basit bildiğimiz bir algoritmanın yeterli olduğunu söyledi. Sözlü olarak "seleksiyon sort gibi" dedi, ama yazdığı döngü komşu elemanları takas eden bir kabarcık sıralamasıdır (bubble sort).

> **Ders sonrası not:** Sözlü adlandırmaya rağmen koddaki sıralama büyük olasılıkla bubble sort'tur. Sonuç için fark etmez, çünkü iki algoritma da ortadaki elemanı doğru yere getirir.

10.4 Derste Çıkan Hatalar

İlk çalıştırmada görüntü bozuk çıktı. Hoca hatayı adım adım aradı:

- Yazma fonksiyonuna verilen boyutlar kontrol edildi, `w` ve `h` yerine `n` ve `m` yazıldı.
- Önceki `min filtre` çağrısı resmin üzerine yazdığı için kapatıldı. Kaynak ve hedef matrislerin bellekte farklı yerlerde olması gerektiği görüldü.
- Sıralamadaki takasta yanlış indis vardı. Bir öğrenci takasın `a[i]` üzerinden yapıldığını, `a[j]` olması gerektiğini söyledi. Bu hata düzeltilmeden çalıştırılınca tuz ve biberler büyümüştü.

Takas `tmp = a[j]`, `a[j] = a[j + 1]`, `a[j + 1] = tmp` biçiminde düzeltilince gürültü kayboldu. Hoca Photoshop Elements'in gürültü filtrelerini de açarak, tuz-biber gürültüsünü kaldırmak için ortanca filtresinin bulunduğunu gösterdi. Yaptığımız işin aynısıdır.

10.5 Kenarlar ve Padding

Resmin ortasında gürültü kalmadı, ama kenarlarda ufak bir şerit göründü. Bu normaldir, çünkü oralar hesaba dahil edilmedi. Programlar bunu şöyle çözer: Resim matrisi büyütülür. Üstten ve alttan birer satır, soldan ve sağdan birer sütun eklenir (padding). Filtre biraz daha geriden başlar ve büyütülmüş resim üzerinde çalışır. Eklenecek satırları olduğu gibi kopyalamak her zaman tam çözüm olmaz, başka teknikler de vardır.

10.6 Filtre Boyutu

Hoca sonra filtreyi 9×9 yaptı. Bu durumda pencerede 81 eleman olur ve ortanca $81/2 = 40$ indisindedir. Bu boyut iyi bir çözüm değildir, iki sebeple:

- **Doku bozulur:** İşin içine çok fazla değer girer ve resmin doğası yine bozulur. Sonuç resim bulanıklaşır, hocanın deyimiyle "ruhsuz" bir hale gelir. Gözün olduğu yeri düşünün: O bölgede geniş pencereyle ortanca almak işe yaramaz. Orijinalliği korumak için filtreyi küçük tutmak gerekir.
- **Yavaşlar:** Resmin genişliği, yüksekliği, pencerenin iki boyutu derken iç içe dört döngü döner. Üstelik her pikselde bir de sıralama vardır. Program hissedilir biçimde geç bitti.

  Hocanın vurgusu: Çoğu uygulamada genellikle 3×3 kullanılır.

11. Resmi Küçültme

Hoca son olarak konuyu sayısal analiz dersine bağlayacağını söyleyerek resmi küçültmeye (shrink) geçti. Yeni matris eski resmin yarı boyutlarında oluşturulur ve `malloc` ile ayrılır. Hoca, iki boyutlu dinamik bellek ayırmanın gelecek dönem görüleceğini, şimdilik bilinmesinin gerekmediğini, güncel dillerde bunun daha kolay olduğunu söyledi.

Algoritma çok basittir: Birer piksel atlayarak, yani `i` ve `j`'yi 2'şer artırarak okuruz. Yeni matrisin indisleri `mm` ve `nn` birer birer ilerler. Her satır bitince `mm` artar, `nn` sıfırlanır. Aşağıda bellek ayırma, derste satır içinde yazılan kod tekrar etmesin diye ayrı fonksiyona alındı.

```
int **matris_ayir(int satir, int sutun)
{
    int i;
    int **b = (int **) malloc(satir * sizeof(int *));
    for (i = 0; i < satir; i++) {
        b[i] = (int *) malloc(sutun * sizeof(int));
    }
    return b;
}

void resim_kucult(int **resim, int n, int m, int q)
{
    int **b = matris_ayir(n / 2, m / 2);
    int i, j, mm = 0, nn = 0;
    char yeni_resim_adi[] = "lena_shrink.pgm";

    for (i = 0; i < n; i += 2) {
        for (j = 0; j < m; j += 2) {
            b[mm][nn] = resim[i][j];
            nn++;
        }
        mm++;
        nn = 0;
    }
    resim_yaz(yeni_resim_adi, b, n / 2, m / 2, q);
}
```

Ders sonrası not: Bu kod büyük olasılıkla yalnız n ve m çift sayı iken doğrudur. Tek sayıda `i` döngüsü $n/2$'den bir fazla tur döner ve ayrılan belleğin dışına büyük olasılıkla taşar. Derste bu sorun görülmedi.

12. Resmi Büyütme

Hoca dersin son sorusunu sordu: Resmi $2n \times 2m$ boyutuna büyütmek istersek ne yaparız?

12.1 Kopyalama

Bir öğrencinin önerisiyle ilk deneme şöyle yapıldı: `i` ve `j` birer birer ilerler, her piksel yeni matriste hem `(mm, nn)` hem de `(mm + 1, nn + 1)` hücresine yazılır. İki hücreye yazıldığı için `mm` ve `nn` ikişer artırılır. Bir öğrenci de bunu hatırlattı. Küçültmeden kalan `n / 2` boyutlu bellek değiştirilmeden çalıştırılınca bellek hatası alındı. Boyutlar `2 * n` ve `2 * m` yapılınca program çalıştı.

```
void resim_buyut_kopya(int **resim, int n, int m, int q)
{
    int **b = matris_ayir(2 * n, 2 * m);
    int i, j, mm = 0, nn = 0;
    char yeni_resim_adi[] = "lena_extended.pgm";

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            b[mm][nn] = resim[i][j];
            b[mm + 1][nn + 1] = resim[i][j];
            nn += 2;
        }
        mm += 2;
        nn = 0;
    }
    resim_yaz(yeni_resim_adi, b, 2 * n, 2 * m, q);
}
```

Resim büyüdü, ama karanlık ve kötü bir görüntü oldu. Gürültüsü giderilmiş ortanca sonucu büyütülünce de düzelmedi. Hoca bunu şöyle açıkladı: Aynı değeri kopyalamak resmin uyumunu (harmonisini) bozan şeyi ortaya çıkarır. Örneğin fotoğrafta gözün rengi var. Onun yanına kopyalanan piksel, olması gereken ten rengi yerine yine gözün rengidir. Yaklaşınca satranç tahtası gibi görünür.

> **Ders sonrası not:** Hoca kopyalamayı önce "interpolasyon", sonra "ekstrapolasyon gibi" diye adlandırdı. Bu kod büyük olasılıkla ikisi de değildir, yalnız kopyalamadır. Görüntünün kararması da büyük olasılıkla harmoniden çok eksik doldurmadan kaynaklanır: `(mm, nn + 1)` ve `(mm + 1, nn)` hücrelerine hiçbir şey yazılmaz. Yeni matrisin hücrelerinin yarısı boş kalır ve boş hücreler satranç tahtası desenini oluşturur. `malloc` ile ayrılan bellek de büyük olasılıkla sıfırlanmış olmak zorunda değildir.

12.2 Komşularla Ortalama

Çözüm olarak hoca bir hücreye değeri aynen, öbür hücreye ise o piksel ile sağdaki pikselin ortalamasını yazdı. Sonra daha iyisi olarak sağdaki, alttaki ve çaprazdaki pikselle dördünün ortalamasını aldı. Bu dersin son kodu buydu.

`i + 1` ve `j + 1` kullanılınca resmin sonunda ayrılan belleğin dışına taşılır. Hoca, dinamik ayrılan bu bellek işletim sisteminden özel izinle alındığı için dışına taşmaya bazı durumlarda izin verilmediğini söyledi. Bu yüzden aşağıdaki kodda döngü bir eksiğine kadar döner (ara adım eklendi).

```
void resim_buyut_ortalama(int **resim, int n, int m, int q)
{
    int **b = matris_ayir(2 * n, 2 * m);
    int i, j, mm = 0, nn = 0;
    char yeni_resim_adi[] = "lena_extended_4.pgm";

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < m - 1; j++) {
            b[mm][nn] = resim[i][j];
            b[mm + 1][nn + 1] = (resim[i][j] + resim[i][j + 1]
                                 + resim[i + 1][j] + resim[i + 1][j + 1]) / 4;
            nn += 2;
        }
```

```
            mm += 2;
            nn = 0;
        }
        resim_yaz(yeni_resim_adi, b, 2 * n, 2 * m, q);
}
```

İki pikselli denemede görüntü biraz farklılaştı, dört pikselli denemede biraz daha farklılaştı. Ama tam bir çözüm değildir.

12.3 Daha İyi Yöntem: Fonksiyon Üretmek

Daha güzel çözüm, resmin içinde belli noktalardan bir interpolasyon yapmaktır.

Soru: Son yaptığımıza bisection diyebilir miyiz? Yani alıp ikiye bölüyoruz. **Cevap:** Bisection diye bir interpolasyon yok. İkisini alıp ortalamasını almak doğru, ama bundan daha iyisi resmin dokusuna, içeriğine bakmaktır.

Hocanın anlattığı yöntem şudur: Bulunduğumuz yerden 20 piksel öteye kadar olan değerleri alalım. Örneğin resimde bir şapka var ve oradan başka bir yere giderken renk değişimi belli bir yol izliyor. Bu 20 sayıdan bir fonksiyon üretelim. Fonksiyon kuadratik, kübik ya da başka bir biçimde olabilir. Sonra bu fonksiyondan ara değerleri okuyup 20 sayının arasına 20 değer daha ekleyelim. Böylece hem resim büyür hem de doku bozulmaz. Bu, sayısal analiz dersindeki interpolasyonla bağlantıdır.

Soru: Dörde dörtlük yapınca her pikselde aynı renk görünmez mi? **Cevap:** Hayır. Burada kopyalama değil, oradan üretilen fonksiyonla ara değerleri doldurma söz konusudur.

Soru: `(mm, nn)` ve `(mm + 1, nn + 1)` yerine `(mm + 1, nn)` ve `(mm, nn + 1)` hücreleri de doldurulsa daha iyi olmaz mı? **Cevap:** Evet, çevredeki diğer hücreler de doldurulup bütün alan iki katına çıkarılabilir. Bu da eklenebilir. Hoca kodu paylaşacağını, isteyenlerin üzerinde değişiklik yapabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Bu yöntemler uygulamada büyük olasılıkla şu adlarla geçer: pikseli komşu hücrelere aynen kopyalamak en yakın komşu (nearest neighbor), dört komşunun ağırlıklı ortalaması çift doğrusal (bilinear), kübik fonksiyon kullanmak çift kübik (bicubic) interpolasyondur.

13. Ayna Görüntüsü

Hoca ayna görüntüsünü (mirror) öğrencilere bıraktı. Mirror yapılırsa resimde sol tarafa bakan kişi sağ tarafa bakar, geri kalan her şey aynı kalır. Bunun matris işlemleriyle kolayca yapılabileceğini söyledi.

> **Ders sonrası not:** Hoca mirror yapmanın matrisi 90 veya 60 derece döndürmek demek olduğunu söyledi. Oysa ayna görüntüsü büyük olasılıkla her satırın ters çevrilmesidir (yatay yansıma), döndürme değildir. Hocanın "sol tarafa bakan kişi sağa bakar" tarifi de bununla uyumludur. 60 derece gibi açılar için tam sayı indisleri yetmez, büyük olasılıkla interpolasyon da gerekir.

Dersin Toparlaması

Bu derste, önceki derslerden kalan örnekler tamamlanarak dönemde görülen araçlar bir araya getirildi. Sepet analizinde üçlüleri bulmak için üç boyutlu matris kullanıldı ve döngü sayısı arttıkça karmaşıklığın n^4'e, 10'lu gruplarda n^{11}'e gittiği görüldü. Üst sınırın gerçek çalışma süresiyle aynı şey olmadığı, süre karşılaştırmasının ise aynı koşullarda defalarca ölçümle yapılması gerektiği vurgulandı. Kumarbaz simülasyonu ise deney sayısı arttıkça sonucun bir olasılığa oturduğunu ve bu örnekte kasanın kazandığını gösterdi.

Görüntü tarafında ortalama filtresinin gürültüyü de topladığı, 0 ve 255'i tamamen atmanın gerçek dokuyu bozduğu, sıralayıp ortadaki değeri alan ortanca filtresinin ise tuz-biber gürültüsünü temiz biçimde giderdiği görüldü. Resim boyutlandırmada küçültme için atlayarak örnekleme, büyütme için komşu ortalamaları denendi. Daha iyi sonuçlar için sayısal analiz

dersindeki interpolasyona işaret edildi. Dinamik bellek ayırma gelecek dönemde, Big-O'nun yanındaki Omega ve Teta gösterimleri ise ilgili derslerde ayrıntılı görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.